

# Relatório Técnico

## Implementação do Protocolo Go-Back-N em Java via UDP

**Disciplina:** Redes de Computadores

**Professor:** Flávio Barbieri Gonzaga

**Aluno:** Jorran Luka Andrade dos Santos

**Aluno:** Pedro Henrique de Almeida

## 1. Introdução

Este relatório descreve o projeto e a implementação do protocolo Go-Back-N (GBN) em Java, utilizando exclusivamente sockets UDP da API padrão do JDK. O objetivo central é demonstrar, de forma prática, como a confiabilidade na transferência de dados pode ser construída pela camada de aplicação sobre um protocolo de transporte não confiável, reproduzindo fielmente as Máquinas de Estado Finitas (FSMs) descritas por Kurose & Ross no Capítulo 3 de *Redes de Computadores: Uma Abordagem Top-Down* (8ª edição).

O sistema é composto por dois módulos independentes: Emissor e Receptor capazes de transferir arquivos arbitrários (imagens, PDFs, executáveis) com controle de fluxo por janela deslizante, simulação de perda de pacotes e verificação de integridade via hash MD5.

## 2. Decisões de Projeto

### 2.1 Estrutura de Classes

O projeto foi organizado em três classes principais, cada uma com responsabilidade bem definida:

- **Packet.java:** Representa um datagrama GBN. Centraliza os processos de serialização (utilizando `ByteBuffer`), desserialização de cabeçalhos e payloads, além de realizar a verificação de integridade.
- **Emissor.java:** Implementa de forma estrita a FSM (Máquina de Estados Finitas) do emissor GBN. É responsável pelo gerenciamento da janela deslizante, execução da thread dedicada para escuta de ACKs, controle do temporizador único e retransmissão em lote.
- **Receptor.java:** Implementa a FSM correspondente ao receptor GBN. Tem como papel aceitar pacotes estritamente dentro da ordem sequencial correta, simular perdas aleatórias com base nas taxas configuradas e realizar a gravação estável do arquivo final em disco.

### 2.2 Formato do Datagrama

O cabeçalho foi definido com 15 bytes fixos, acrescentando um campo de checksum (4 bytes) além do mínimo sugerido pelo enunciado, garantindo robustez na detecção de corrupção de pacotes. A estrutura detalhada dos campos segue abaixo:

- **tipo (1 byte):** Identifica a função do pacote. Os valores definidos são 0 para DATA, 1 para ACK, 2 para HANDSHAKE e 3 para FIN.
- **num\_seq (4 bytes):** Representa o número de sequência atribuído ao pacote para controle de ordenação e envio.
- **num\_ack (4 bytes):** Número de confirmação cumulativa (utilizado exclusivamente nos pacotes do tipo ACK).
- **tamanho\_dados (2 bytes):** Indica a quantidade exata de bytes válidos contidos no campo de payload.
- **checksum (4 bytes):** Valor da soma de verificação calculada sobre o cabeçalho e o payload para assegurar a integridade.
- **dados (Até 1024 bytes):** Payload correspondente aos bytes brutos do arquivo em transferência.

## 2.3 Espaço de Numeração de Sequência

O espaço de sequência foi estabelecido seguindo a regra fundamental  $\text{maxSeqNum} = \text{windowSize} * 2$ . Esta definição cumpre estritamente a restrição clássica do algoritmo Go-Back-N, mitigando por completo eventuais ambiguidades de interpretação entre pacotes novos e retransmissões dentro de um mesmo ciclo da janela de transmissão.

Como exemplo prático de aplicação, ao adotar uma janela de tamanho  $N = 8$ , determina-se que  $\text{maxSeqNum} = 16$ . Consequentemente, o intervalo lógico de numeração varia de 0 a 15 de forma circular ao longo de toda a execução do fluxo.

## 2.4 Gerenciamento do Temporizador

O GBN utiliza um único temporizador, sempre associado ao pacote mais antigo enviado e não confirmado (base). A classe `java.util.Timer` foi escolhida pela sua simplicidade e precisão suficiente para o timeout de 500 ms adotado. A lógica segue estritamente a FSM: ao receber um ACK que avança a base, o timer é cancelado e reiniciado apenas se ainda houver pacotes pendentes de confirmação ( $\text{base} \neq \text{nextSeqNum}$ ). Em caso de timeout, todos os pacotes no intervalo  $[\text{base}, \text{nextSeqNum} - 1]$  são retransmitidos em lote e o timer é reiniciado.

## 2.5 Sincronização entre Threads

O Emissor executa duas threads concorrentes: a thread principal (responsável pelo envio de dados via `rdtSend`) e a thread de ACKs (`receiveAcks`). As variáveis compartilhadas `base`, `nextSeqNum` e o buffer de pacotes são protegidas de forma síncrona pelo monitor da

instância (synchronized(this)). O mecanismo wait() / notifyAll() é usado para bloquear a thread de envio quando a janela está cheia e desbloqueá-la assim que novos ACKs forem validados.

## 2.6 Handshake e Encerramento (FIN)

Antes da transferência dos dados, o Emissor realiza uma conexão inicial enviando um pacote HANDSHAKE contendo: probabilidade de perda (float, 4 bytes), tamanho total do arquivo (long, 8 bytes), tamanho da janela (int, 4 bytes) e caminho de destino (String UTF-8). O Receptor confirma com um ACK correspondente e prepara a recepção dos blocos. Ao término do envio, o Emissor despacha um pacote FIN carregando o hash MD5 do arquivo original (16 bytes). O Receptor responde com um FIN-ACK, valida a integridade final e encerra a escrita em disco. Ambos os processos de handshake e encerramento implementam retransmissão persistente com até 10 tentativas antes de abortar.

## 2.7 Verificação de Integridade MD5 (R9)

O hash MD5 é calculado nativamente no Emissor antes do início da transferência por meio da classe java.security.MessageDigest. O digest resultante é transmitido de forma segura no payload do pacote FIN. O Receptor recalcula o MD5 com base nos dados gravados e gera uma comparação direta; o resultado (OK ou FALHA) é exibido na tela de estatísticas.

# 3. Dificuldades Encontradas

Durante o ciclo de desenvolvimento do projeto, foram mapeadas e solucionadas as seguintes complexidades técnicas:

- **Race condition (Thread Concorrência):** O Emissor utiliza duas threads executando simultaneamente: a thread principal, responsável pelo envio dos pacotes, e uma thread assíncrona dedicada à recepção de ACKs. Como ambas acessavam e modificavam as variáveis compartilhadas de controle da janela (base e nextSeqNum), diferentes ordens de execução podiam gerar inconsistências no estado da janela deslizante, ocasionando avanços incorretos e comportamento imprevisível do protocolo.  
**Solução:** Proteção completa de todas as seções críticas e operações de leitura/escrita utilizando o monitor nativo do Java por meio de blocos
- **Instabilidade por ACKs Atrasados/Duplicados:** Devido ao uso do UDP e ao mecanismo de retransmissão por timeout, ACKs referentes a transmissões anteriores podiam chegar após o avanço da janela ou serem recebidos mais de uma vez. Sem uma validação adequada, essas confirmações obsoletas poderiam ser interpretadas como válidas, provocando reinicializações desnecessárias do temporizador e disparos incorretos do

algoritmo de retransmissão.

**Solução:** Implementação de uma validação lógica baseada em intervalos numéricos circulares, garantindo que apenas ACKs pertencentes à janela de transmissão atualmente pendente fossem aceitos.

- **Pacotes de Encerramento (FIN) Duplicados:** Como o protocolo utiliza retransmissão do pacote FIN para garantir o encerramento da comunicação, datagramas de controle atrasados ou retransmitidos ainda podiam chegar ao Receptor após a finalização lógica da sessão. Isso ocasionava processamento desnecessário de mensagens de encerramento e podia interferir no fechamento adequado do socket.

**Solução:** Implementação de uma janela temporal de espera passiva durante o encerramento do Receptor, permitindo absorver e descartar pacotes remanescentes antes do fechamento definitivo da comunicação.

## 4. Testes Realizados

### 4.1 Metodologia

Os testes foram realizados em ambiente de rede local (loopback 127.0.0.1), isolando o comportamento e validando o correto funcionamento da simulação de perdas de pacotes configurada no Receptor. Três categorias de arquivos foram empregadas: uma imagem JPEG (10 KB), um documento PDF (94 KB) e um Executável binário (~1,06 MB). Como a perda de pacotes é simulada no Receptor via sorteio aleatório (`Math.random()`, sem seed fixa) a cada pacote recebido em ordem, cada execução de um mesmo cenário produz uma amostra diferente do processo. Para mitigar essa variabilidade estocástica, cada cenário (combinação de arquivo, janela N e probabilidade de perda p) foi repetido em 5 execuções independentes, reportando-se a média de retransmissões e de tempo de transferência.

### 4.2 Teste Básico Sem Perda

O teste inicial ocorreu com `prob_perda = 0.0` e janela `N = 4`. Em todos os casos, as transferências foram bem-sucedidas e os hashes MD5 bateram perfeitamente com os originais.

| Arquivo | Tamanho | Janela N | Perda (%) | Retransmissões | MD5 |
|---------|---------|----------|-----------|----------------|-----|
| Imagem  | 10 KB   | 4        | 0%        | 0              | OK  |

| Arquivo    | Tamanho  | Janela N | Perda (%) | Retransmissões | MD5 |
|------------|----------|----------|-----------|----------------|-----|
| Pdf.pdf    | 94 KB    | 4        | 0%        | 0              | OK  |
| Executável | ~1,06 MB | 4        | 0%        | 0              | OK  |

### 4.3 Teste com Simulação de Perdas

Ativando as perdas aleatórias artificiais, a robustez das retransmissões em rajada do GBN foi atestada. A taxa efetiva de perda aproximou-se estatisticamente do valor configurado à medida que o volume total de pacotes crescia (conforme a Lei dos Grandes Números), o Executável, com 1082 pacotes, apresentou taxas efetivas de 9,30% e 9,16% para N=4 e N=8 respectivamente, ambas muito próximas dos 10% configurados. Todos os valores abaixo são médias de 5 execuções independentes por cenário. Na coluna "Perda Observada", a imagem JPEG (apenas 10 pacotes) é reportada em contagem absoluta de perdas simuladas, pois uma amostra tão pequena torna a porcentagem estatisticamente irrelevante; os demais arquivos são reportados em porcentagem por terem amostras suficientes.

Um resultado relevante observado nos dados: janelas maiores (N=8) produziram consistentemente mais retransmissões do que janelas menores (N=4) sob a mesma probabilidade de perda, por exemplo, 102,0 vs. 39,2 retransmissões no PDF e 869,0 vs. 444,0 no Executável. Isso é esperado pelo comportamento do GBN: quando um pacote é perdido, o protocolo retransmite todos os pacotes em voo na janela ativa. Com N=8, uma única perda pode forçar até 8 retransmissões; com N = 4, no máximo 4. Portanto, o custo de retransmissão do GBN escala com o tamanho da janela, o que evidencia sua principal desvantagem em relação ao protocolo de Repetição Seletiva.

| Arquivo | Tamanho | Janela N | Perda Config. | Perda Observada           | Retransmissões     | MD5 |
|---------|---------|----------|---------------|---------------------------|--------------------|-----|
| Imagem  | 10 KB   | 4        | 10%           | 0,6 perdas (méd. 5 exec.) | 2,2 (méd. 5 exec.) | OK  |
| Imagem  | 10 KB   | 8        | 10%           | 0,6 perdas                | 3,5 (méd.          | OK  |

| Arquivo    | Tamanho  | Janela N | Perda Config. | Perda Observada          | Retransmissões          | MD5       |
|------------|----------|----------|---------------|--------------------------|-------------------------|-----------|
|            |          |          |               | (méd. 5 exec.)           | 5 exec.)                |           |
| Pdf        | 94 KB    | 4        | 10%           | 9,41%<br>(méd. 5 exec.)  | 39,2 (méd. 5 exec.)     | <b>OK</b> |
| Pdf        | 94 KB    | 8        | 10%           | 12,04%<br>(méd. 5 exec.) | 102,0<br>(méd. 5 exec.) | <b>OK</b> |
| Executável | ~1,06 MB | 4        | 10%           | 9,30%<br>(méd. 5 exec.)  | 444,0<br>(méd. 5 exec.) | <b>OK</b> |
| Executável | ~1,06 MB | 8        | 10%           | 9,16%<br>(méd. 5 exec.)  | 869,0<br>(méd. 5 exec.) | <b>OK</b> |

## 5. Análise dos Resultados - Impacto do Tamanho da Janela N (R8)

A tabela abaixo apresenta o desempenho detalhado de tempo e vazão na transferência do arquivo Executável (~1,06 MB) para diferentes tamanhos de janela N, mantendo a probabilidade de perda fixa em 10%. Cada valor de retransmissões corresponde a 1 execução (N=1, N=2, N=4, N=8 e N=16).

| Janela N | Pacotes de Dados | Retransmissões | Tempo Aprox. (s) | Throughput Est. (Mbit/s) |
|----------|------------------|----------------|------------------|--------------------------|
| 1        | 1082             | 118            | 59,85 s          | 0,15                     |
| 2        | 1082             | 226            | 58,06 s          | 0,15                     |
| 4        | 1082             | 444            | 54,01 s          | 0,16                     |
| 8        | 1082             | 869            | 56,48 s          | 0,16                     |
| 16       | 1082             | 2128           | 68,07 s          | 0,13                     |

## 5.1 Análise

Os resultados revelam um comportamento diferente do esperado pela teoria clássica: em ambiente loopback, o aumento de N não melhorou proporcionalmente o throughput. O melhor desempenho foi obtido com N=4 (54,01 s, 0,16 Mbit/s). Com N = 1 e N = 2 os tempos foram similares entre si (59,85 s e 58,06 s), confirmando que janelas muito pequenas limitam o pipeline. A partir de N=8 (56,48 s) e N=16 (68,07 s) o desempenho oscilou, com N=16 gerando 2128 retransmissões mais que o dobro das 869 de N=8 sem ganho proporcional de tempo.

Esse comportamento é explicado pela natureza do GBN em loopback: a latência de rede é praticamente zero, então janelas maiores não trazem ganho de pipeline. Em compensação, cada perda força a retransmissão de toda a janela ativa com N=16, uma única perda dispara até 16 retransmissões imediatas. O overhead de retransmissão supera o benefício do paralelismo, degradando o tempo total. O ponto ótimo observado foi N = 4, onde o equilíbrio entre aproveitamento da janela e custo de retransmissão produziu o menor tempo de transferência.

## 5.2 Impacto da Probabilidade de Perda

| Prob. de Perda (p) | Janela N | Retransmissões Aprox. | Tempo Aprox. (s) | Throughput   | Eficiência Estimada |
|--------------------|----------|-----------------------|------------------|--------------|---------------------|
| 0%                 | 8        | 0                     | 0,74 s           | 11,98 Mbit/s | 100,0%              |
| 5%                 | 8        | 466                   | 30,26 s          | 0,29 Mbit/s  | 69,9%               |
| 10%                | 8        | 869                   | 71,79 s          | 0,12 Mbit/s  | 56,9%               |
| 20%                | 8        | 2195                  | 140,05 s         | 0,06 Mbit/s  | 33,0%               |
| 30%                | 8        | 3763                  | 238,19 s         | 0,04 Mbit/s  | 22,3%               |

Os dados confirmam a degradação severa do GBN sob perda de pacotes. Sem perda ( $p = 0\%$ ), o throughput atingiu 11,98 Mbit/s com tempo de 0,74 segundos, o protocolo operou no limite do loopback. Com apenas 5% de perda, o throughput despencou para 0,29 Mbit/s e o tempo subiu para 30,26 s: uma queda de 97,6% no throughput causada por apenas 466 retransmissões. A progressão é acentuada:  $p = 10\%$  gerou 869 retransmissões e 71,79 s;  $p = 20\%$  chegou a 2195 retransmissões e 140,05 s;  $p = 30\%$  atingiu 3763 retransmissões e 238,19 s. A eficiência (pacotes úteis / total enviados) caiu de 100% para 22,3% em  $p = 30\%$ . Isso ilustra a principal desvantagem estrutural do GBN: uma única perda força a retransmissão de todos os pacotes em voo na janela ativa (até  $N = 8$  pacotes), multiplicando o custo de cada evento de perda. Em comparação, o protocolo de Repetição Seletiva retransmitiria apenas o pacote efetivamente perdido, mantendo eficiência muito superior sob as mesmas taxas de perda.

## 6. Conclusão

A implementação do protocolo Go-Back-N em Java sobre UDP atingiu plenamente todos os objetivos acadêmicos e técnicos propostos. A transferência robusta de dados binários de naturezas e tamanhos variados foi assegurada pela correspondência idêntica das somas de verificação MD5 em 100% dos cenários avaliados, superando taxas de erro artificial de até 30%.

Os experimentos práticos confirmaram os fundamentos teóricos: janelas maiores



aumentam o aproveitamento do canal em redes com latência significativa, mas apresentam retornos decrescentes em loopback local. Em contrapartida, sob taxas elevadas de perda, o comportamento de retransmissão em rajada do GBN degrada severamente a eficiência evidenciando a vantagem do protocolo de Repetição Seletiva em enlaces com alta taxa de descarte, onde retransmitir apenas o pacote perdido seria muito mais eficiente.

Em última análise, a atividade consolidou de forma robusta os conceitos de base da disciplina de redes de computadores, cobrindo a modelagem formal de FSMs, controle de fluxo elástico via buffers deslizantes, gestão precisa de temporizadores assíncronos e serialização compacta em baixo nível através de objetos ByteBuffer.